Python Program for Blocks World

Blocks World is a classic Artificial Intelligence planning problem. Blocks can be stacked on one another or placed on a table. Only a clear (top) block can be moved at a time.

Problem

Initial state: B is on A, and C is on the table.
Goal state: A is on the table, B is on A, and C is on B.

Python Code

```
from collections import deque

# Each stack is written from bottom to top.
# ("A", "B") means B is on top of A.

def make_state(stacks):
    """Sort stacks so equivalent arrangements have the same representation."""
    return tuple(sorted(tuple(stack) for stack in stacks if stack))


def get_moves(state):
    """Generate legal moves. Only a top (clear) block can be moved."""
    moves = []

    for source_index, source in enumerate(state):
        block = source[-1]

        # Move the top block onto the table.
        if len(source) > 1:
            new_stacks = [list(stack) for stack in state]
            new_stacks[source_index].pop()
            new_stacks.append([block])
            moves.append((
                make_state(new_stacks),
                f"Move {block} from its stack to the table"
            ))

        # Move the top block onto another stack's top block.
        for dest_index, destination in enumerate(state):
            if source_index == dest_index:
                continue

            new_stacks = [list(stack) for stack in state]
            new_stacks[source_index].pop()
            new_stacks[dest_index].append(block)
            moves.append((
                make_state(new_stacks),
                f"Move {block} onto {destination[-1]}"
            ))

    return moves


def solve_blocks_world(initial, goal):
    """Use Breadth-First Search to find a shortest plan."""
    start = make_state(initial)
    target = make_state(goal)

    queue = deque([start])
    parent = {start: (None, None)}

    while queue:
        current = queue.popleft()

        if current == target:
            actions = []
            while parent[current][0] is not None:
                previous, action = parent[current]
                actions.append(action)
                current = previous
            return list(reversed(actions))

        for next_state, action in get_moves(current):
            if next_state not in parent:
                parent[next_state] = (current, action)
                queue.append(next_state)
```

```
    return None


# Initial: B is on A; C is on the table.
initial_state = [["A", "B"], ["C"]]

# Goal: A is on the table; B is on A; C is on B.
goal_state = [["A", "B", "C"]]

plan = solve_blocks_world(initial_state, goal_state)

print("BLOCKS WORLD - SOLUTION")
print("Initial state:", initial_state)
print("Goal state:   ", goal_state)
print()

if plan is None:
    print("No solution found.")
else:
    for step, action in enumerate(plan, start=1):
        print(f"Step {step}: {action}")
    print("\nGoal state reached!")
```

Expected Output

```
BLOCKS WORLD - SOLUTION
Initial state: [['A', 'B'], ['C']]
Goal state:    [['A', 'B', 'C']]

Step 1: Move C onto B

Goal state reached!
```

Important Concepts

Concept	Meaning
State	The current arrangement of the blocks.
Initial state	The arrangement from which the problem starts.
Goal state	The required final arrangement.
Operator / Action	A legal move, such as moving a clear block onto another clear block or onto the table.
Breadth-First Search (BFS)	Explores states level by level and finds a solution with the fewest moves when all moves have equal cost.

Note: Change `initial_state` and `goal_state` to test other arrangements using the same block names.